H1: Werken met Visual Studio

Labo 1 · week 1 · installatie en je eerste project

SEMESTER 1 · BASIS PROGRAMMEREN

Vandaag

1. **Welkom**: wat leer je dit semester, en hoe verloopt een labo?
2. **Programmeren**: wat is een algoritme, en wat is C#?
3. **Installatie** van Visual Studio
4. **Je eerste project**, met de juiste instellingen
5. **Afspraken**: naamgeving en minpunten
6. **Fouten** in je code herkennen en verbeteren



Welkom! Vandaag installeren we Visual Studio. Neem gerust een koffie, de installatie duurt toch even.

Welkom

Wat leer je dit semester?

Week	Hoofdstuk
1	H1: Werken met Visual Studio
2	H2: Variabelen en datatypes
3	H3: Strings en hun methoden
4	H4: Beslissingen
5 en 6	H5: Loops
7	toetsweek
8 en 9	H6: Arrays en lists
10	H7: Methoden
11	H9: Meerdimensionaal werken
12	H10: Gevorderde tekstverwerking
13	recap en vragen

😈 *Dertien weken programmeren. Tegen week 5 droom je in accolades.*

Hoe verloopt een labo?

- Eerst **korte theorie** met slides, zoals deze.
- Daarna ga je zelf aan de slag met de **oefeningen**. Daar leer je het meest.
- Alles staat op de cursussite **csharp.apload.be**: de planning, de theorie, de slides en de oefeningen.
- Bij elke oefening staat een **oefening-assistent** die je helpt als je vastzit, zonder de oplossing voor te zeggen.

Wat heb je nodig? Een **laptop** met **Visual Studio 2026**.



De oefening-assistent helpt je dag en nacht. Ik niet, ik slaap ook eens.

Wat is programmeren?

Een programma volgt een **algoritme**: een reeks instructies, in een vaste volgorde. Het “recept”.

```
1. Haal de dop van het ventiel
2. Plaats de pomp op het ventiel
3. Begin te pompen
```

Een andere volgorde geeft vreemde (en soms fatale) fouten. Een computer doet **exact** wat je zegt, ook als het onzin is.



Eerst pompen en dan pas de dop eraf? Dat is een algoritme om lucht te verspillen.

C# en .NET

- **C#** spreek je uit als *"siesjarp"*.
- C# is een **programmeertaal**: exact en ondubbelzinnig, anders begrijpt de computer het niet.
- Het hoort bij **.NET** (*"dotnet"*), een omgeving van Microsoft uit 2000.
- Met .NET maak je webapplicaties, desktopapplicaties, apps, games en nog veel meer.



Niet "C hekje" zeggen. Tenzij je de lector wil zien huiveren.

Installatie

Editor en SDK

- Een **editor**, zoals Visual Studio, helpt je bij het **schrijven** van code: kleuren, suggesties en foutmeldingen.
- De **SDK** (*software development kit*) bevat alles om je code ook echt **uit te voeren**.
- Met Visual Studio installeer je beide in één keer.



Code schrijven in Kladblok kan ook. Je kan ook met een lepel een tunnel graven.

Visual Studio 2026 Community installeren

1. Download via **visualstudio.microsoft.com/downloads** en kies de **Community**-versie.
2. Duid de workload **.NET desktop development** aan.
3. Bij **Individual components**: vink **.NET 8.0 Runtime** en **.NET 10.0 Runtime** aan.
4. Klik op **Install**.
5. Log in met je account van de hogeschool: **s-nummer@ap.be**.

> ⚠ **WAARSCHUWING**
>
> **IntelliCode** moet **uitgeschakeld** zijn. Die AI-hulp schrijft code voor je, en dan leer je het zelf niet.

Visual Studio installeren: het enige programma waarbij de laadbalk zelf een loop lijkt.

Je eerste project

Eerst een goede plek

- Maak een **rootmap** voor al je vakken, bijvoorbeeld **AP**.
- Maak daarin een map **Basis Programmeren**. Al je projecten van dit vak komen daar.

> ⚠️ **WAARSCHUWING**
>
> Zet je rootmap **niet** in OneDrive, Dropbox of Google Drive. Automatisch synchroniseren geeft problemen, zeker in vakken met Git.



Je code in OneDrive: wat kan er misgaan? Alles. Alles kan misgaan.

Een nieuw project

1. **Create a new project** → kies **Console App** (C#).
2. Geef een goede **Project name** in PascalCase, bijvoorbeeld `HelloWorld`. Kies de juiste **Location**.
3. Kies als **Framework .NET 8.0**.
4. Vink **“Do not use top-level statements” aan**.
5. Klik op **Create**.

> **BELANGRIJK**
>
> Vergeet je stap 4, dan kost dat op een evaluatie **5 punten**.

Vijf punten aftrek voor één vinkje. Het duurste vinkje van je opleiding.

Nog één instelling

Rechtsklik op je project in de **Solution Explorer** → **Properties** → **Global Usings** → **General**: zet **Implicit global usings uit**.

Daarna moet je zelf bovenaan `using System;` schrijven. Anders:

```
1  Console.WriteLine("Hello, World!");  // ❌ CS0103
```

> **OPMERKING**
>
> The name ‘Console’ does not exist in the current context

Zonder `using System;` kent C# zelfs `Console` niet. Zoals je eerste dag op een nieuwe school.

Hello, World!

```
 1  using System;
 2
 3  namespace HelloWorld
 4  {
 5      internal class Program
 6      {
 7          static void Main(string[] args)
 8          {
 9              Console.WriteLine("Hello, World!");
10          }
11      }
12  }
```

Starten met **F5** of **Ctrl + F5**:

```
Hello, World!
```

"Hello, World!": sinds 1972 het eerste wat elke programmeur zegt. Sociaal contact begint klein.

Afspraken

Namen geven

Afspraken maken je code leesbaar, voor anderen en voor jezelf over een paar maanden.

- **PascalCase**: elk woord begint met een hoofdletter: `HelloWorld`, `MyFirstMethod`. Voor projecten, klassen en methoden.
- **camelCase**: zoals PascalCase, maar het eerste woord met een kleine letter: `firstName`, `numberOfStudents`. Voor variabelen.
- Geen spaties, en namen in het **Engels**. Enkel tekst die de gebruiker ziet, schrijf je in het Nederlands.



camelCase heeft bulten, zoals een kameel. Wie zonder bulten schrijft, krijgt minpunten.

Minpunten op evaluaties

Wat	Aftrek
top-level statements gebruiken	-5
een methode in een andere methode definiëren	-3
`goto`, en `break` of `continue` buiten een `switch`	-3
omslachtige, redundante code	tot -3
naamgeving niet volgens de afspraken	-2
slordige bladspiegel (inspringen, uitlijnen)	-1
code die niet compileert	-1

 *`goto` staat op de zwarte lijst. Zelfs de compiler fronst.*

Hoe lees je een oefening?

Elke oefening heeft dezelfde opbouw: een **titel**, een **toelichting**, soms een **vertrekpunt** en een **voorbeeldinteractie**.

In een voorbeeldinteractie is een regel met `>` iets wat **de gebruiker** intypt:

```
Wat is je naam?
> Tim
Dag Tim!
```

Lees de opgave twee keer. Dan moet je de code maar één keer schrijven.

Fouten in je code

Opzettelijk een fout maken

```
1  Console.WriteLine("Hello, World!")
```

De puntkomma op het einde is weg.

- Er verschijnt een **rode kronkellijn** waar de fout zit.
- De **Error List** toont de fout, met het **regelnummer** en de **kolom**: CS1002 `; expected`.
- Zolang er een fout in je code staat, kan je programma **niet starten**.



Een puntkomma vergeten: de klassieker van elk programmeursleven. Ook na twintig jaar.

Zoek de fout 🤔

Code	Fout
`Console.WriteLine("Hallo")`	puntkomma vergeten (CS1002)
`Console.RaedLine();`	schrijffout: `ReadLine` (CS0117)
`console.WriteLine("Hallo");`	hoofdletter: `Console` (CS0103)
`Console.Readline();`	hoofdletter: `ReadLine` (CS0117)

De compiler is een strenge spellingscorrector. Zonder gevoel voor humor.

Eén fout, drie meldingen

```
1  Console.WriteLine("Hallo);
```

- CS1010: Newline in constant
- CS1026:) expected
- CS1002: ; expected

Eén vergeten aanhalingsteken, en de compiler raakt de draad kwijt. **Begin altijd bij de eerste fout**: de rest verdwijnt vaak vanzelf.

😈 *Eén aanhalingsteken vergeten, drie foutmeldingen. Een kettingreactie, zoals dominostenen.*

Samengevat

- Een **algoritme** is een reeks instructies in een vaste volgorde.
- **Visual Studio 2026 Community** met **.NET desktop development**, IntelliCode **uit**.
- Nieuw project: **Console App**, **.NET 8.0**, **“Do not use top-level statements”** aan, **implicit global usings** uit.
- Namen in het **Engels**: **PascalCase** voor klassen en methoden, **camelCase** voor variabelen.
- Een fout? Lees de **Error List** en begin bij de **eerste** fout.



Je hebt vandaag een programma gemaakt dat “Hello, World!” zegt. De wereld zegt nog niets terug.

Aan de slag 💻

Volg de stappen bij de oefeningen van **H1**, “Structuur van je oefeningen”:

1. Maak het project **LaboOefeningen** (instructies op DigitAP)
2. Maak een klasse **EenProgrammaSchrijvenInCSharp**
3. Maak je eerste methode **MyFirstMethod**
4. Roep die methode op vanuit **Main**
5. Kuis op, zodat je klaar bent voor de volgende oefeningen

👉 Klaar? Kijk alvast naar de oefening **MyFirstProgram**. Volgende les leer je alles wat je ervoor nodig hebt.

👹 `MyFirstMethod`: *je eerste methode. Hopelijk niet je laatste.*